Studio Nx

JS 297 — WAV 인코딩

2026-10-20

JavaScript 297 — WAV 인코딩 (`audio/wav.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/wav.js`
작업	S1-11b WAV 출력: 렌더링한 소리를 WAV 파일 바이트로 바꿉니다 (FR-303)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 296 화면 연결, 다음 298 내보내기 계획 계산

1. 하는 일

채널별 `Float32Array`(값은 -1에서 1)를 받아 WAV 파일의 바이트(`Uint8Array`)로 만듭니다. 16bit, 24bit, 32bit float를 지원하고, 정규화에 쓸 피크 계산과 배율 계산도 여기에 있습니다. 소리 장치를 전혀 쓰지 않는 순수 함수라서 Node에서도 시험합니다.

```
렌더링 결과 [왼쪽 Float32Array, 오른쪽 Float32Array]
        │  peakOfChannels ─▶ 피크 ─▶ normalizeGain ─▶ 배율 (정규화를 켰을 때만)
        ▼
   encodeWav(채널들, 샘플레이트, 비트 깊이, { gain, dither })
        ▼
   [RIFF 머리][fmt 조각][(fact 조각)][data 조각: 프레임마다 왼쪽, 오른쪽 번갈아]
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `startExport` (300)	렌더링이 끝나면 피크를 재고, 정규화 배율을 구하고, WAV 바이트를 만들어 다운로드에 넘깁니다.
`wavByteLength`	내보내기 창의 "예상 파일 크기"를 렌더링 전에 계산합니다.
시험	`export_unit.mjs`가 머리와 값을, `render_offline.mjs`가 브라우저의 디코더로 다시 읽은 값을 확인합니다.

3. 코드

3.1 피크와 정규화 배율

```
export function peakOfChannels(channels) {
  let peak = 0;
  for (let c = 0; c < channels.length; c++) {
    const data = channels[c];
    for (let i = 0; i < data.length; i++) {
      const v = data[i] < 0 ? -data[i] : data[i];
      if (v > peak) peak = v;
    }
  }
```

```
  return peak;
}
```

```
export function normalizeGain(peak, targetDb) {
  if (!(peak > 0)) return 1;
  return Math.pow(10, targetDb / 20) / peak;
}
```

3.2 파일 크기

```
export function wavByteLength(frames, numChannels, bitDepth) {
  const header = bitDepth === 32 ? 58 : 44;   // 32bit float 는 fmt 확장(2바이트)과 fact 조각(12바이트)이
더 있음
  return header + frames * numChannels * (bitDepth / 8);
}
```

3.3 인코딩

```
export function encodeWav(channels, sampleRate, bitDepth, options) {
  const opt = options || {};
  const gain = opt.gain != null ? opt.gain : 1;
  const dither = opt.dither == null ? true : !!opt.dither;
  if (BIT_DEPTHS.indexOf(bitDepth) < 0) throw new Error('지원하지 않는 비트 깊이: ' + bitDepth);
  const numCh = channels.length;
  const frames = numCh > 0 ? channels[0].length : 0;
  const bytes = bitDepth / 8;
  const isFloat = bitDepth === 32;
  const total = wavByteLength(frames, numCh, bitDepth);
  const buffer = new ArrayBuffer(total);
  const view = new DataView(buffer);

  // 머리
  let p = 0;
  writeText(view, p, 'RIFF'); view.setUint32(p + 4, total - 8, true); writeText(view, p + 8, 'WAVE'); p
+= 12;
  writeText(view, p, 'fmt '); view.setUint32(p + 4, isFloat ? 18 : 16, true);
  view.setUint16(p + 8, isFloat ? 3 : 1, true);                  // 1 = PCM, 3 = IEEE float
  view.setUint16(p + 10, numCh, true);
  view.setUint32(p + 12, sampleRate, true);
  view.setUint32(p + 16, sampleRate * numCh * bytes, true);      // 초당 바이트
  view.setUint16(p + 20, numCh * bytes, true);                   // 한 프레임의 바이트
  view.setUint16(p + 22, bitDepth, true);
  p += 24;
  if (isFloat) {
    view.setUint16(p, 0, true); p += 2;                          // 확장 크기 0
    writeText(view, p, 'fact'); view.setUint32(p + 4, 4, true); view.setUint32(p + 8, frames, true); p +
= 12;
  }
  writeText(view, p, 'data'); view.setUint32(p + 4, frames * numCh * bytes, true); p += 8;

  // 데이터 (채널을 프레임마다 번갈아 씀)
  let seed = 22222;                                              // 디더 난수는 고정 시드: 같은 입력이면
같은 파일
  const rand = function () { seed = (seed * 1664525 + 1013904223) >>> 0; return seed / 4294967296; };
```

```
  for (let i = 0; i < frames; i++) {
    for (let c = 0; c < numCh; c++) {
      const v = channels[c][i] * gain;
      if (isFloat) {
        view.setFloat32(p, v, true); p += 4;
      } else if (bitDepth === 16) {
        let s = v * 32767;
        if (dither) s += rand() - rand();                        // TPDF: 최대 ±1 LSB
        s = Math.round(s);
        if (s > 32767) s = 32767; else if (s < -32768) s = -32768;
        view.setInt16(p, s, true); p += 2;
      } else {
        let s = Math.round(v * 8388607);
        if (s > 8388607) s = 8388607; else if (s < -8388608) s = -8388608;
        if (s < 0) s += 16777216;                                // 24bit 음수는 2의 보수
        view.setUint8(p, s & 255); view.setUint8(p + 1, (s >> 8) & 255); view.setUint8(p + 2, (s >> 16)
& 255); p += 3;
      }
    }
  }
  return new Uint8Array(buffer);
}
```

4. WAV 파일의 모양

위치(바이트)	내용	16bit와 24bit	32bit float
0 ~ 11	`RIFF`, 파일 크기 - 8, `WAVE`	같음	같음
12 ~ 35	`fmt ` 조각: 형식, 채널, 샘플레이트, 초당 바이트, 프레임 바이트, 비트	형식 1(PCM), 조각 크기 16	형식 3(IEEE float), 조각 크기 18
이어서	`fact` 조각: 프레임 수	없음	있음 (12바이트)
이어서	`data` 조각: 소리	머리 44바이트	머리 58바이트

한 프레임은 채널 수만큼의 샘플이 이어진 것입니다. 스테레오는 왼쪽, 오른쪽, 왼쪽, 오른쪽 순서로 씁니다.

5. 최신 문법으로 쓰면

```
// peakOfChannels 와 normalizeGain 을 최신 문법으로 쓴 모양
export const peakOfChannels = (channels) =>
  channels.reduce((peak, data) => data.reduce((p, v) => Math.max(p, Math.abs(v)), peak), 0);

export const normalizeGain = (peak, targetDb) =>
  peak > 0 ? 10 ** (targetDb / 20) / peak : 1;

// encodeWav 의 옵션 읽기
export function encodeWav(channels, sampleRate, bitDepth, { gain = 1, dither = true } = {}) { /* ... */
}
```

전통 문법 (실제 코드)	최신 문법
겹친 `for` 문으로 최댓값 찾기	`reduce` 두 번
`Math.pow(10, x)`	`10 ** x`

전통 문법 (실제 코드)	최신 문법
`opt.gain != null ? opt.gain : 1`	기본값 구조 분해 `{ gain = 1 }`
`function () { ... }`	`() => ...`

6. 왜 이렇게

선택	이유
정규화는 배율 `gain`만 넘기고 채널은 건드리지 않습니다	몇 분짜리 곡은 채널 배열이 수백 MB입니다. 정규화하려고 배열을 한 번 더 복사하지 않고, 파일에 쓰는 순간 곱합니다.
정규화 배율은 렌더링 뒤의 피크로 구합니다	이펙트 체인을 거친 최종 소리의 가장 큰 값이 목표 dBFS가 되도록 하는 것이 정규화의 정의입니다. 소리가 전혀 없으면(피크 0) 배율을 1로 두어 0으로 나누지 않습니다.
16bit에는 TPDF 디더를 넣습니다	32bit float로 계산한 소리를 16bit로 줄이면 작은 소리의 끝이 계단처럼 거칠어집니다(양자화 잡음). 최대 ±1 LSB의 아주 작은 잡음을 섞으면 이 거친 느낌이 부드러운 잡음으로 바뀝니다. FL Studio도 16bit 출력에 디더 옵션을 둡니다.
디더의 난수는 고정 시드의 선형 합동 생성기입니다	같은 소리를 두 번 내보내면 같은 파일이 나와야 시험하기 쉽고, 문제가 생겼을 때 재현할 수 있습니다. 소리가 거칠어지지 않는다는 목적에는 시드가 무엇이든 상관없습니다.
24bit와 32bit에는 디더를 넣지 않습니다	24bit는 양자화 잡음이 이미 들리지 않을 만큼 작고, 32bit float는 줄이는 과정이 없습니다.
32bit는 정수가 아니라 부동소수점(IEEE float)입니다	내부 계산과 같은 형식이라 값이 그대로 남고, 1을 넘는 값도 잘리지 않습니다. 이어서 믹싱이나 마스터링을 할 때 쓰기 좋습니다. FL Studio도 같은 이유로 32bit float를 둡니다.
32bit 머리에는 `fact` 조각과 확장 크기를 넣습니다	표준은 PCM이 아닌 형식에 이 두 가지를 요구합니다. 일부 프로그램은 없으면 파일을 열지 못합니다. 브라우저의 디코더는 둘 다 읽는 것을 시험에서 확인했습니다.
정수 값은 범위를 잘라서(클리핑) 씁니다	마스터가 0dBFS를 넘으면 값이 정수 범위를 벗어납니다. 잘라 두지 않으면 부호가 뒤집혀 큰 잡음이 납니다.
24bit는 바이트 세 개를 직접 씁니다	`DataView`에는 24bit 쓰기 함수가 없습니다. 음수는 2의 보수(16777216을 더한 값)로 바꿔 아래쪽 바이트부터 씁니다.
파일 크기를 `wavByteLength`로 따로 둡니다	같은 계산을 인코더와 창의 크기 안내가 함께 쓰므로, 안내한 크기와 실제 크기가 어긋나지 않습니다. 시험에서 둘이 같은 것을 확인합니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 WAV 부분(38건)이 머리와 값을 확인합니다. 16bit의 길이, `RIFF`와 `WAVE`와 `data` 글자, 파일 크기 필드, 형식 번호(1), 채널과 샘플레이트와 비트 필드, 초당 바이트와 프레임 바이트, 0.5가 16384, 1.0이 32767인지 봅니다. 2와 -2는 32767과 -32768로 잘립니다. 디더는 같은 입력이면 같은 파일이고, 끄면 값이 한결같으며, 켜면 값이 흔들리되 2 LSB 안에 있습니다. 24bit는 0.5가 4194304이고, 32bit float는 형식 번호 3과 `fact` 조각을 갖고 값이 그대로이며 1.5도 그대로입니다.
- `docs/handoff/s1-11b_tests/render_offline.mjs`의 J 부분(11건)은 실제 렌더링 결과를 세 가지 비트 깊이로 인코딩하고, 브라우저의 `decodeAudioData`로 다시 읽어서 길이, 샘플레이트, 채널 수, 파일 크기, 값을 비교합니다. 16bit는 최대 차이가 0.0002 이하, 24bit는 0.000001 이하, 32bit는 거의 0입니다. 정규화를 켜면 피크가 -0.3dBFS(0.966)가 됩니다.

- 시험 중 발견한 일: 처음에 두 건이 실패했습니다. -0.5를 16bit로 바꾼 값이 -16384가 아니라 -16383이었습니다. 자바스크립트의 `Math.round`가 정확히 .5인 값을 항상 위쪽(+방향)으로 올리기 때문입니다. 소리에는 영향이 없는 1 LSB 차이라서 코드는 두고, 시험의 기대값을 고치고 이유를 주석으로 남겼습니다.